SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Juhi Surin	**Reg. No.:**	192525148
Date:	14-08-2026	**Duration:**	60 minutes

Code of Conduct

I __Juhi Surin (192525148)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __Juhi Surin___

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
   - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ARCHITECTURE DESIGN REPORT

CO3 Assessment Tool 1 – Software Engineering (CSA10)

Assigned Problem Statement (Q37):

"AI-Based Digital Waste Recycling Marketplace and Resource Recovery System"

Submitted by:	Juhi Surin
Registration Number:	192525148
Course:	CSA10 – Software Engineering
Faculty In-Charge:	Dr. R. Latha
Department:	B.Tech AIML
Date of Submission:	14 August 2026

1. Problem Analysis

1.1 Industry Context

Rapid urbanization and industrial growth have led to a sharp rise in municipal and industrial waste generation across cities. Governments, municipal corporations, and environmental organizations are actively promoting circular-economy practices to reduce landfill dependency and improve resource recovery. Advances in Artificial Intelligence, the Internet of Things (IoT), cloud computing, and digital marketplace platforms now make it possible to coordinate waste collection, automatically segregate recyclable material, and connect multiple stakeholders in the recycling value chain in near real time.

1.2 Problem Statement

Existing waste management systems largely operate in silos: waste generators (households and industries), recycling facilities, and waste collection agencies have no unified platform to coordinate collection, classification, and trading of recyclable material. This fragmentation results in poor recycling rates, inefficient collection routes, and an increased volume of reusable material ending up in landfills. The assigned project therefore proposes an AI-powered Digital Waste Recycling Marketplace and Resource Recovery System that:

- Connects households, industries, recyclers, and waste-collection agencies on a single digital platform.
- Automatically classifies recyclable materials (plastic, paper, metal, e-waste, organic, glass) using an AI/computer-vision model.
- Optimizes waste-collection schedules and routes for collection agencies.
- Recommends nearby recycling centers based on material type and geolocation.
- Facilitates the trading (listing, bidding, and purchase) of recyclable materials between generators and recyclers.
- Generates sustainability analytics and dashboards to track resource recovery and environmental impact.

1.3 Project Objectives

1. Digitize recyclable waste trading between generators and recyclers.
2. Classify waste materials automatically using an AI/ML model.
3. Connect waste generators (households/industries) with recyclers and collection agencies.
4. Optimize waste collection schedules and vehicle routing.
5. Monitor recycling activities and transaction history in real time.
6. Generate sustainability analytics for stakeholders and administrators.
7. Promote circular-economy practices among users.
8. Reduce the volume of recyclable waste that is sent to landfills.

1.4 Functional Requirements

ID	Functional Requirement	Description
FR-01	User Registration & Roles	Support registration/login for households, industries, recyclers, collection agencies, and administrators with role-based access.
FR-02	Waste Listing	Allow generators to list recyclable material with photos, quantity, and category.
FR-03	AI Waste Classification	Automatically classify uploaded waste images into material categories using a trained AI model.
FR-04	Marketplace & Trading	Enable recyclers to browse, bid on, and purchase listed recyclable material.
FR-05	Collection Scheduling	Optimize and assign pickup schedules/routes for collection agencies.
FR-06	Recycling Center Locator	Recommend the nearest suitable recycling center using geolocation and material type.
FR-07	Notifications	Notify users of pickup status, bid updates, and transaction confirmations.
FR-08	Sustainability Analytics	Provide dashboards showing recycling volumes, landfill diversion, and CO2 impact.
FR-09	Transaction & Payment Records	Maintain a secure, auditable record of all trades and payments.
FR-10	Admin Monitoring	Allow administrators to monitor platform activity and manage users/listings.

1.5 Expected Outcomes

9. Increased recycling rates through easier discovery and trading of recyclable material.
10. Reduced environmental pollution from unmanaged waste.
11. Improved waste-collection efficiency through optimized scheduling and routing.
12. Better resource recovery via AI-assisted classification and marketplace matching.
13. Enhanced collaboration between households, industries, recyclers, and agencies.
14. Reduced dependency on landfills for disposal.
15. A sustainable, technology-driven waste-management ecosystem.
16. Increased public participation and awareness in recycling activities.

1.6 High-Level System Architecture

The system is designed as a set of loosely coupled, independently deployable services so that each concern (marketplace logic, AI classification, logistics, analytics) can be developed, versioned, and containerized separately. Figure 1 illustrates the overall architecture.

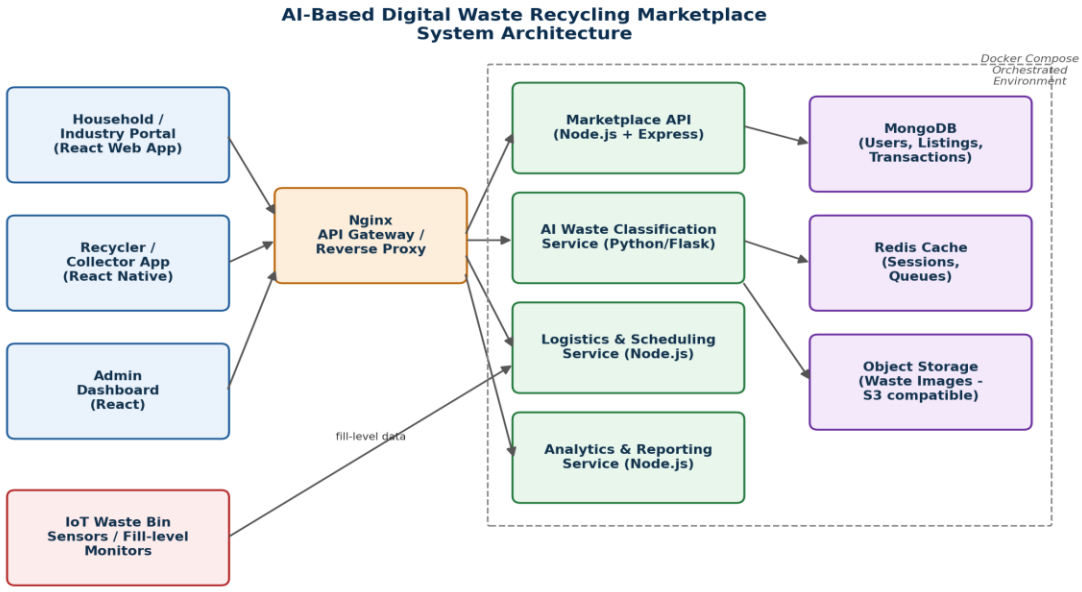

Figure 1: High-level system architecture of the AI-Based Digital Waste Recycling Marketplace

2. Project Structure Design

2.1 Repository Folder Structure

The repository is organized by microservice so that each component has an independent codebase, dependency file, and Dockerfile, which keeps version control history clean and allows each service to be built and deployed on its own.

```
waste-recycling-marketplace/
|-- .git/                    # Git internal metadata (auto-created by git init)
|-- .gitignore               # Files/folders excluded from version control
|-- README.md                # Project overview, setup & usage instructions
|-- LICENSE                  # Project license
|-- docker-compose.yml       # Multi-container orchestration definition
|-- .env.example             # Sample environment variables (no secrets)
|-- docs/
|   |-- architecture-diagram.png
|   |-- git-branching-diagram.png
|   \-- api-specification.md
|-- frontend/                # React web application
|   |-- Dockerfile
|   |-- package.json
|   |-- public/
|   \-- src/
|       |-- components/
|       |-- pages/
|       \-- services/
|-- backend/                 # Node.js + Express marketplace API
|   |-- Dockerfile
|   |-- package.json
|   \-- src/
|       |-- routes/
|       |-- controllers/
|       |-- models/
|       \-- middleware/
|-- ai-service/              # Python Flask AI waste-classification microservice
```

```
|   |-- Dockerfile
|   |-- requirements.txt
|   |-- model/
|   |   \-- waste_classifier.h5
|   \-- app.py
|-- logistics-service/          # Node.js collection-scheduling microservice
|   |-- Dockerfile
|   |-- package.json
|   \-- src/
|-- nginx/                      # Reverse proxy / API gateway configuration
|   |-- Dockerfile
|   \-- nginx.conf
\-- tests/
    |-- unit/
    \-- integration/
```

Figure 2: Project repository folder structure

2.2 Justification of Major Folders and Files

Folder / File	Purpose / Justification
.gitignore	Prevents build artefacts, node_modules/, __pycache__/, .env, and log files from polluting the repository and exposing secrets.
README.md	Onboarding document describing setup, architecture, and run instructions so new contributors can start quickly.
docker-compose.yml	Central place that defines and wires together every containerized service, network, and volume for local/production deployment.
.env.example	Documents required environment variables without committing actual secrets, keeping configuration reproducible and secure.
frontend/	Isolated React codebase for the customer-facing marketplace portal; independently buildable and containerized.
backend/	Node.js/Express service holding marketplace business logic, REST APIs, and database models.
ai-service/	Python microservice hosting the trained waste-classification model behind a lightweight Flask API.
logistics-service/	Handles collection scheduling and route optimization, kept separate so it can scale independently of the marketplace API.
nginx/	Single entry point (API gateway/reverse proxy) that routes client requests to the correct backend service and terminates SSL.
docs/	Stores architecture and workflow diagrams referenced in the technical report, keeping documentation versioned alongside code.
tests/	Automated unit and integration tests, ensuring each commit can be validated before merging into develop/main.

3. Git Repository Initialization

3.1 Steps to Initialize the Repository

The following sequence of commands was used to create and configure the Git repository for the project:

```
# 1. Create the project folder and move into it
mkdir waste-recycling-marketplace && cd waste-recycling-marketplace

# 2. Initialize an empty Git repository
git init

# 3. Configure author identity (once per machine)
git config user.name "Student Name"
git config user.email "student@saveetha.ac.in"

# 4. Create the essential base files
touch README.md .gitignore LICENSE

# 5. Stage and create the first (initial) commit
git add .
git commit -m "chore: initial commit - project scaffold and base files"

# 6. Link the local repository to a remote (e.g., GitHub/GitLab)
git remote add origin https://github.com/<org>/waste-recycling-marketplace.git

# 7. Rename default branch and push
git branch -M main
git push -u origin main
```

3.2 Purpose of Essential Git Files

File / Folder	Purpose
.git/	Hidden metadata directory created by 'git init'; stores the full object database, commit history, branches, and configuration for the repository.
.gitignore	Lists files and folders (e.g., node_modules/, __pycache__/, .env, *.log, build/) that Git should never track, keeping the repository clean and secrets safe.
README.md	First point of reference for any contributor; documents purpose, setup steps, environment variables, and run/deploy instructions.
LICENSE	States the legal terms under which the source code may be used, modified, and distributed.
.gitattributes	Normalizes line endings and marks specific paths (e.g., generated files) for consistent handling across operating systems.

4. Git Branching Strategy

4.1 Selected Branching Model: Gitflow

Because the project is composed of multiple independently deployable microservices (frontend, backend API, AI service, logistics service) developed by a team over several sprints, a Gitflow-based branching strategy was adopted. It separates in-progress work from stable, releasable code and gives every change type (feature, release, hotfix) a clear, predictable home.

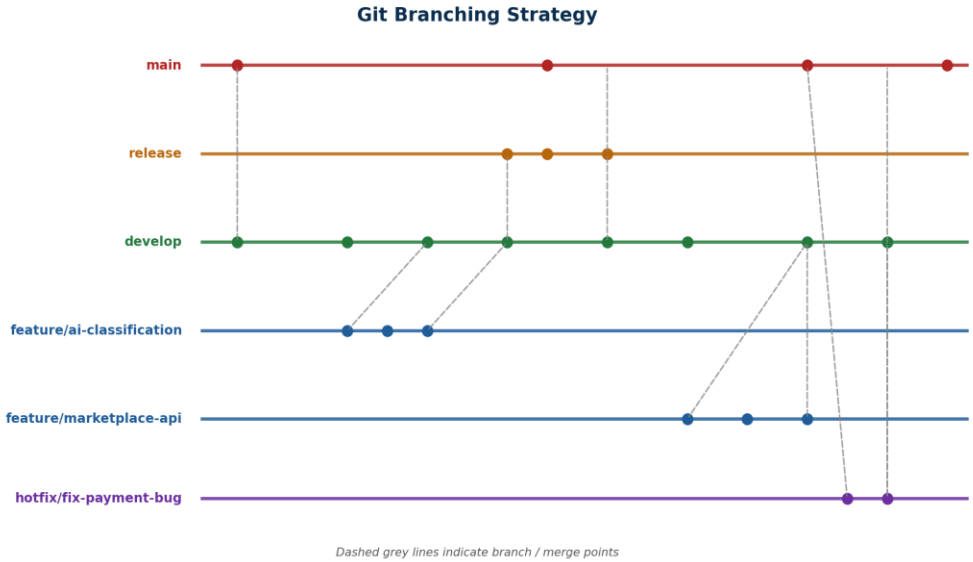

Figure 3: Git branching strategy used for the project (Gitflow model)

4.2 Branch Roles

Branch	Purpose
main	Always reflects the latest production-ready, deployed state of the application. Only release and hotfix branches merge here.
develop	Integration branch where completed features are merged and tested together before a release is cut.
feature/*	One branch per feature (e.g., feature/ai-classification, feature/marketplace-api), branched from develop and merged back into develop once complete and reviewed.
release/*	Created from develop when preparing a new version; used for final testing, version bumps, and documentation before merging into main and develop.
hotfix/*	Branched directly from main to urgently patch production issues (e.g., a payment bug), then merged into both main and develop.

4.3 Justification for the Assigned Project

Gitflow is well suited to this project for several reasons:

- Multiple parallel workstreams — AI classification model integration, marketplace API, and logistics scheduling — can each progress in isolated feature branches without blocking one another.
- The develop branch gives the team a stable integration point to test how the AI service, backend API, and frontend interact before anything is released.

- Because the platform involves financial trading transactions, release branches allow rigorous testing and staged sign-off before code reaches main/production.
- Hotfix branches ensure that critical issues (e.g., a payment or scheduling bug) can be patched in production immediately without pulling in unfinished develop work.
- The model scales cleanly as more contributors and microservices are added to the platform.

5. Version Control Workflow

The table below demonstrates the Git workflow performed for the project, including meaningful commits, branch creation, merging, conflict resolution, and repository synchronization, together with the purpose of each operation.

#	Git Operation	Purpose / Explanation
1	git checkout -b feature/ai-classification develop	Create and switch to a new feature branch off develop to build the AI classification module in isolation.
2	git add ai-service/ && git commit -m "feat(ai-service): add waste image classification endpoint"	Stage the AI service code and record a meaningful, scoped commit.
3	git push -u origin feature/ai-classification	Publish the feature branch to the remote so it can be reviewed and backed up.
4	git checkout develop && git merge --no-ff feature/ai-classification	Merge the completed feature back into develop, preserving branch history with --no-ff.
5	git checkout -b feature/marketplace-api develop	Start a second, parallel feature branch for the marketplace trading API.
6	git merge develop (inside feature/marketplace-api)	Synchronize the feature branch with the latest develop changes before finishing work, surfacing a conflicting change to package.json.
7	(edit conflicting hunks) → git add package.json && git commit	Manually resolve the merge conflict by keeping both dependency additions, then complete the merge commit.
8	git checkout develop && git merge --no-ff feature/marketplace-api	Merge the resolved feature branch into develop.
9	git checkout -b release/1.0.0 develop	Cut a release branch from develop to prepare version 1.0.0 for production.
10	git checkout main && git merge --no-ff release/1.0.0 && git tag v1.0.0	Merge the tested release into main and tag the production release.
11	git checkout develop && git merge --no-ff release/1.0.0	Back-merge the release branch into develop so both branches stay in sync.
12	git fetch origin && git pull origin develop	Synchronize the local repository with the latest remote changes from teammates.
13	git push origin main develop --tags	Push all updated branches and the release tag to the shared remote repository.

5.1 Conflict Resolution Example

While merging the latest develop branch into feature/marketplace-api, both branches had added different dependencies to the same section of backend/package.json, producing a merge conflict. Git marked the conflicting region as follows:

```
<<<<<<< HEAD
  "axios": "^1.6.0",
=======
  "express-validator": "^7.0.1",
>>>>>>> develop
```

The conflict was resolved by manually editing the file to retain both dependencies, then the resolution was staged and committed to complete the merge:

```
  "axios": "^1.6.0",
  "express-validator": "^7.0.1",

git add backend/package.json
git commit -m "merge: resolve dependency conflict between develop and feature/marketplace-api"
```

6. Commit History Analysis

6.1 Sample Commit History (git log --oneline)

Commit	Message	Date
a1c3f9e	chore: initial commit - project scaffold and base files	Aug 03, 2026
b7d2a41	feat(frontend): scaffold React app with routing and base layout	Aug 04, 2026
c48e0aa	feat(backend): initialize Express server and MongoDB connection	Aug 05, 2026
d59f3bc	feat(ai-service): add waste image classification endpoint	Aug 07, 2026
e6a0c12	test(ai-service): add unit tests for classification accuracy threshold	Aug 07, 2026
f7b1d34	feat(backend): implement waste listing and marketplace bidding APIs	Aug 09, 2026
089c2e5	fix(backend): correct pagination bug in listing search endpoint	Aug 10, 2026
19ad6f7	merge: resolve dependency conflict between develop and feature/marketplace-api	Aug 11, 2026
2bce881	feat(logistics-service): add collection route optimization logic	Aug 12, 2026
3cf9a02	docs: update README with setup and environment variable instructions	Aug 13, 2026
4d0ab73	chore(docker): add Dockerfiles for backend, ai-service, and frontend	Aug 13, 2026
5e1bc94	feat(deploy): add docker-compose.yml for full-stack orchestration	Aug 14, 2026
6f2cda5	release: prepare v1.0.0 - version bump and changelog	Aug 15, 2026

6.2 Commit Message Conventions

All commit messages follow the Conventional Commits format: type(scope): short description, for example feat(ai-service): add waste image classification endpoint. This practice contributes to project maintenance in several ways:

- The type prefix (feat, fix, docs, chore, test, refactor, merge, release) immediately communicates the intent of a change without opening the diff.
- The scope (e.g., backend, ai-service, docker) narrows down which part of the multi-service system was affected, which is essential in a microservice repository.
- Small, focused commits (one logical change each) make git bisect and code review far more effective when isolating the source of a bug.
- A consistent history enables automatic changelog generation and simplifies onboarding, since new contributors can read the log as a narrative of the project's evolution.
- Clear messages make rollback (git revert <commit>) safe and predictable, since each commit's blast radius is well understood.

7. Docker Environment Design

7.1 Why Docker Is Suitable for This Project

The AI-Based Digital Waste Recycling Marketplace is built as a set of polyglot microservices — a Node.js API, a Python AI model, a React frontend, and supporting data stores — each with different runtimes and dependencies. Docker is well suited here because it:

- Packages each service with its exact runtime and dependencies, eliminating "works on my machine" inconsistencies between developer laptops, CI, and production.
- Allows the Node.js backend, Python AI microservice, and React frontend — each with different language runtimes — to run side-by-side without conflicting system dependencies.
- Provides process isolation, so a crash or memory spike in the AI classification service cannot directly affect the marketplace API container.
- Enables horizontal scaling of individual services (e.g., running multiple AI-service replicas during peak image-upload traffic) independent of the rest of the stack.
- Makes local development, staging, and production environments reproducible through a single docker-compose.yml definition.
- Simplifies CI/CD, since a built image is the same artifact that is tested and deployed, removing environment drift.

7.2 Components Requiring Containerization

Component	Base Image	Reason for Containerization
frontend	node:20-alpine (build) -> nginx:alpine (serve)	Isolates the React build toolchain from the runtime web server; produces a small static-serving image.
backend (marketplace API)	node:20-alpine	Isolates Node.js/Express runtime, its npm dependencies, and connection configuration.
ai-service	python:3.11-slim	Bundles the ML/AI libraries (TensorFlow/Keras, Pillow, NumPy) and trained model with the exact Python version they were validated against.
logistics-service	node:20-alpine	Runs scheduling/route-optimization logic independently so it can scale separately from the core marketplace API.
nginx (gateway)	nginx:alpine	Provides a single reverse-proxy entry point that routes to each backend service and can terminate TLS.
mongodb	mongo:7	Official, versioned image ensures a consistent, disposable database instance for development and testing.
redis	redis:7-alpine	Lightweight in-memory store for session/cache data, isolated from the application containers.

8. Dockerfile Development

8.1 Dockerfile — Backend (Node.js Marketplace API)

A multi-stage build is used: the first stage installs dependencies and prepares the app, and the second stage copies only the production artefacts into a slim runtime image, keeping the final image small and reducing the attack surface.

```
# --- Stage 1: install dependencies & build ---
FROM node:20-alpine AS builder
WORKDIR /usr/src/app

# Copy dependency manifests first to leverage Docker layer caching
COPY package*.json ./
RUN npm ci --omit=dev

# Copy application source
COPY . .

# --- Stage 2: minimal runtime image ---
FROM node:20-alpine
WORKDIR /usr/src/app
ENV NODE_ENV=production

# Run as a non-root user for security
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=builder /usr/src/app ./
USER appuser

EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:5000/health || exit 1

CMD ["node", "src/server.js"]
```

8.2 Dockerfile — AI Waste Classification Service (Python)

```
FROM python:3.11-slim
WORKDIR /app

# System dependencies required by image-processing libraries
RUN apt-get update && apt-get install -y --no-install-recommends \
    libglib2.0-0 libsm6 libxext6 libxrender1 && \
    rm -rf /var/lib/apt/lists/*

# Install Python dependencies first (cache-friendly)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy trained model and application code
COPY model/ ./model/
COPY app.py .

EXPOSE 6000
HEALTHCHECK --interval=30s --timeout=10s CMD python -c "import requests; requests.get('http://localhost:6000/health')" || exit 1

CMD ["python", "app.py"]
```

8.3 Purpose of Key Dockerfile Instructions

Instruction	Purpose
FROM	Selects the base image (runtime + OS layer) the container is built from, e.g. node:20-alpine or python:3.11-slim.
WORKDIR	Sets the working directory inside the container for all subsequent instructions and the default runtime CWD.
COPY	Copies files/folders from the build context into the image; manifests are copied before source code to maximize Docker layer-cache reuse.
RUN	Executes a command during the image build (e.g., npm ci, pip install) and bakes the resulting filesystem changes into a new layer.
ENV	Defines environment variables (e.g., NODE_ENV=production) available to the process at runtime.
USER	Switches to a non-root user before the container runs, reducing the security risk if the container is compromised.
EXPOSE	Documents the network port the containerized service listens on (informational; actual publishing is done in docker-compose.yml).
HEALTHCHECK	Defines a command Docker uses to periodically verify the container is actually serving requests, enabling automatic restart of unhealthy containers.
CMD	Specifies the default command executed when the container starts (here, launching the Node.js server or Flask app).

9. Docker Compose Design

Docker Compose is used to define and orchestrate all containers required by the platform with a single declarative file, so the entire stack can be brought up with one command.

```
version: "3.9"

services:
  frontend:
    build: ./frontend
    container_name: waste_frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - waste-net

  backend:
    build: ./backend
    container_name: waste_backend
    restart: unless-stopped
    environment:
      - NODE_ENV=production
      - MONGO_URI=mongodb://mongo:27017/waste_marketplace
      - REDIS_URL=redis://redis:6379
      - AI_SERVICE_URL=http://ai-service:6000
    ports:
      - "5000:5000"
    depends_on:
      - mongo
      - redis
      - ai-service
    networks:
      - waste-net

  ai-service:
    build: ./ai-service
    container_name: waste_ai_service
    restart: unless-stopped
    ports:
      - "6000:6000"
    volumes:
      - ai-model-data:/app/model
    networks:
      - waste-net

  logistics-service:
    build: ./logistics-service
    container_name: waste_logistics
    restart: unless-stopped
    environment:
      - MONGO_URI=mongodb://mongo:27017/waste_marketplace
    depends_on:
      - mongo
    networks:
      - waste-net

  nginx:
    build: ./nginx
    container_name: waste_gateway
    ports:
```

```
    - "80:80"
  depends_on:
    - frontend
    - backend
  networks:
    - waste-net

 mongo:
  image: mongo:7
  container_name: waste_mongo
  restart: unless-stopped
  volumes:
    - mongo-data:/data/db
  networks:
    - waste-net

 redis:
  image: redis:7-alpine
  container_name: waste_redis
  restart: unless-stopped
  networks:
    - waste-net

volumes:
 mongo-data:
 ai-model-data:

networks:
 waste-net:
  driver: bridge
```

9.1 Explanation of Configuration

Element	Explanation
services	Each block (frontend, backend, ai-service, logistics-service, nginx, mongo, redis) defines one container, its build context or image, and its runtime configuration.
networking	A single custom bridge network, waste-net, lets containers reach each other by service name (e.g., the backend calls http://ai-service:6000) while remaining isolated from other Docker networks on the host.
volumes	Named volumes mongo-data and ai-model-data persist database files and the trained ML model outside the container lifecycle, so data survives container restarts/rebuilds.
environment variables	Configuration such as MONGO_URI, REDIS_URL, and AI_SERVICE_URL is injected at runtime, keeping connection details out of the source code and easy to change per environment.
depends_on	Ensures containers start in a sensible order (e.g., backend waits for mongo, redis, and ai-service to be created) so dependent services are available when the app starts.
ports	Maps container-internal ports to host ports (e.g., "80:80" for nginx) so the platform is reachable from outside Docker.
restart: unless-stopped	Automatically restarts a service container if it crashes, improving resilience without manual intervention.

10. Container Deployment and Testing

10.1 Deployment Steps

```
# Build images and start every container in the background
docker-compose up --build -d

# Verify all containers are running and healthy
docker-compose ps

# Tail logs of a specific service while testing
docker-compose logs -f backend
```

10.2 Simulated Deployment Output

The following output illustrates the expected result of docker-compose ps after a successful deployment, confirming that every container built from this report's Dockerfiles and docker-compose.yml is up and healthy:

```
NAME              IMAGE                        STATUS                  PORTS
waste_gateway     waste-recycling_nginx        Up 3 minutes (healthy)  0.0.0.0:80->80/tcp
waste_frontend    waste-recycling_frontend     Up 3 minutes            0.0.0.0:3000->80/tcp
waste_backend     waste-recycling_backend      Up 3 minutes (healthy)  0.0.0.0:5000->5000/tcp
waste_ai_service  waste-recycling_ai-service   Up 3 minutes (healthy)  0.0.0.0:6000->6000/tcp
waste_logistics   waste-recycling_logistics    Up 3 minutes            5001/tcp
waste_mongo       mongo:7                      Up 3 minutes            27017/tcp
waste_redis       redis:7-alpine               Up 3 minutes            6379/tcp
```

10.3 Testing Procedure

17. Health-check verification: each service exposes a /health endpoint; docker inspect confirms the HEALTHCHECK status is "healthy" for backend and ai-service.
18. API smoke test: curl -X POST http://localhost/api/waste-listings with a sample JSON payload is used to confirm the backend can reach MongoDB and persist a listing.
19. AI classification test: a sample waste image is POSTed to http://localhost:6000/classify; the endpoint is expected to return a JSON body such as {"category": "plastic", "confidence": 0.94}.
20. Inter-service communication test: creating a listing through the frontend triggers a call from backend to ai-service (via the internal waste-net network) to auto-tag the material category, verifying container networking.
21. Load/resilience test: docker-compose stop ai-service is run to confirm the backend degrades gracefully (returns an "uncategorized" status) instead of crashing, then docker-compose start ai-service restores full functionality.
22. End-to-end UI test: the React frontend at http://localhost:3000 is used to register a user, list a waste item, and confirm it appears in the recycler's marketplace view, validating the full request path through nginx to the backend and database.

10.4 Sample API Test Evidence

```
$ curl -X POST http://localhost/api/waste-listings \
   -H "Content-Type: application/json" \
   -d '{"title":"Mixed PET bottles","quantityKg":25,"imageUrl":"..."}'
```

```
HTTP/1.1 201 Created
{
  "id": "66f1a2...",
  "title": "Mixed PET bottles",
  "category": "plastic",
  "confidence": 0.94,
  "status": "listed"
}
```

11. Benefits of Git and Docker

Adopting Git and Docker together provides significant, complementary benefits across the lifecycle of the AI-Based Digital Waste Recycling Marketplace:

11.1 Collaboration

Feature branches allow the frontend, backend, AI, and logistics teams to work concurrently without overwriting each other's changes, while pull requests and code review on each branch improve code quality before merging into develop.

11.2 Version Management

Every change to the marketplace logic, AI model integration, and infrastructure configuration is tracked with a full audit trail, letting the team trace exactly when and why a feature (e.g., the classification confidence threshold) was changed, and revert safely if needed.

11.3 Portability

Docker images encapsulate each microservice with its exact dependencies, so the platform runs identically on a developer's laptop, the CI runner, and the production server — eliminating environment-specific bugs.

11.4 Deployment

docker-compose up reproduces the entire multi-service stack (frontend, backend, AI service, logistics service, gateway, database, cache) with a single command, drastically reducing deployment time and manual configuration errors.

11.5 Scalability

Because each service is independently containerized, the AI classification service — the most compute-intensive component — can be scaled horizontally (docker-compose up --scale ai-service=3) during peak usage without scaling the rest of the platform.

11.6 Maintainability

Clear commit history, descriptive branch names, and isolated Dockerfiles make it straightforward for new contributors to understand, debug, and extend individual services without needing to understand the entire system at once.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts in shared configuration files (e.g., package.json, docker-compose.yml) when multiple feature branches modified the same section concurrently.
- Large Docker image size for the AI service due to machine-learning libraries, which slowed down local builds and image transfer.
- Keeping environment variables and service URLs consistent across development, testing, and production docker-compose configurations.
- Coordinating the AI classification service's startup time (model loading) with the backend's dependency checks, which occasionally caused early requests to fail before the model finished loading.
- Integrating simulated IoT waste-bin fill-level data into the logistics-service scheduling logic during early development.

12.2 Suggested Improvements and Future Enhancements

- Introduce a CI/CD pipeline (e.g., GitHub Actions) to automatically run tests and build/push Docker images on every merge to develop and main.
- Adopt multi-stage, multi-architecture Docker builds to reduce the AI-service image size and support both amd64 and arm64 deployment targets.
- Add a container orchestration layer such as Kubernetes for production, enabling auto-scaling, rolling updates, and self-healing beyond what Docker Compose offers.
- Implement a proper readiness probe for the AI service so dependent services wait until the model has fully loaded before routing traffic to it.
- Add centralized logging and monitoring (e.g., ELK stack or Prometheus/Grafana) across all containers for better observability in production.
- Extend branch protection rules and mandatory pull-request reviews on main and develop to further strengthen code quality and prevent accidental direct pushes.

13. Conclusion

This report presented the design and implementation approach for an AI-Based Digital Waste Recycling Marketplace and Resource Recovery System, applying software-engineering best practices in version control and containerization. A Gitflow branching strategy was used to manage parallel development of the marketplace API, AI classification service, and logistics service, with meaningful commits and documented conflict resolution demonstrating a professional Git workflow. The application was designed as a set of Docker containers orchestrated with Docker Compose, covering the frontend, backend, AI microservice, logistics service, reverse proxy, database, and cache, with each component's Dockerfile and configuration explained and justified. Together, Git and Docker provide the collaboration, portability, and scalability needed to build and maintain a reliable, production-ready recycling marketplace platform.

References

- Git Documentation — https://git-scm.com/doc
- Docker Documentation — https://docs.docker.com
- Docker Compose File Reference — https://docs.docker.com/compose/compose-file/
- Atlassian Gitflow Workflow Guide — https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow
- Conventional Commits Specification — https://www.conventionalcommits.org
- MongoDB Official Docker Image — https://hub.docker.com/_/mongo